

v1 Hardware Debug & Tooling

Author	Ryan Gaffere
Date	April 22th, 2026
Document Version	v1.0

This document describes the design, operation, and usage of the RG Sonic32 CPU v1 hardware debug and observability tooling.

Overview

The Hardware Debug & Observability Tooling for RG Sonic32 CPU v1 provides a lightweight, passive mechanism for inspecting CPU execution while running on FPGA hardware. It exposes key internal execution events over UART, enabling validation of program behavior directly on the physical system without relying solely on simulation.

The tooling is integrated at the `cpu_top` level and taps into existing CPU signals without modifying functional behavior. It monitors instruction context, writeback activity, store operations, halt detection, and architectural register state. These signals are captured as structured debug events and transmitted as human-readable trace lines over a serial interface.

This tooling enables direct correlation between executed instructions and observed hardware state, providing a practical method for validating CPU correctness on physical hardware.

Role in Validation

The debug and observability tooling serves as the primary mechanism for validating CPU behavior on physical hardware. By exposing execution state in real time, it enables direct verification of instruction execution, data movement, and final architectural state without relying solely on simulation.

During development, the tooling was used to:

- Verify correct instruction execution by correlating program flow with observed PC and instruction values
- Confirm register writeback behavior by inspecting destination registers and associated data
- Validate memory operations through captured store address and data events
- Ensure overall program correctness by comparing final register state after halt with expected results

This approach allows execution traces to be analyzed step-by-step, providing a practical and reliable method for confirming functional correctness of the CPU on FPGA hardware.

How It Works

The debug and observability tooling operates as a passive monitoring layer attached to the CPU datapath and control logic. It captures selected internal signals during execution, converts them into structured debug events, and transmits them over UART for external inspection.

Signal Capture

The module taps into key CPU signals at the `cpu_top` level, including:

- Program Counter (PC)
- Instruction Register (IR)
- Destination register and writeback data
- Memory store address and data
- Control signals such as write enables and halt

These signals are observed without affecting the CPU's timing or functional behavior.

Event Selection

Rather than logging every cycle, the tooling captures discrete execution events that are most useful for validation. These include:

- Register writeback operations
- Memory store operations
- Halt condition

This selective approach reduces unnecessary output while preserving visibility into meaningful state changes.

Event Formatting

When a qualifying event is detected, the captured signals are formatted into a structured trace record. Each record encodes the relevant execution context (e.g., PC, instruction, destination register, data) using fixed-width hexadecimal fields to ensure readability and consistency.

Buffering and Transmission

Formatted events are placed into a small internal buffer (FIFO) to decouple event generation from UART transmission. This allows the system to continue capturing events even when the serial interface is busy.

The buffered data is then streamed out through the UART transmitter as human-readable text. On the host side, this appears as a continuous execution trace that reflects the CPU's behavior in real time.

Execution Process

At a high level, the process is:

1. CPU executes an instruction
2. Debug module observes internal signals
3. If a relevant event occurs, signals are captured
4. Event is formatted into a trace record
5. Record is queued in the buffer
6. UART transmits the record to the host

Output Format

The debug tooling emits execution trace records as human-readable text over UART. Each line corresponds to a single captured event and encodes the relevant execution context using fixed-width hexadecimal fields.

A typical trace line is shown below:

```
PC=FFFF0000 IR=0480000A RD=04 WB=0000000A
```

Field Description

Each field represents a specific component of CPU state at the time of the event:

- **PC (Program Counter):** Address of the instruction being executed.
- **IR (Instruction Register):** Encoded instruction word currently being processed.

- **RD (Destination Register):** Register index receiving the writeback value.
- **WB (Writeback Data):** Data written to the destination register.

Event Variations

The exact format of each trace line depends on the type of event captured.

Register Writeback

```
PC=FFFF0000 IR=0480000A RD=04 WB=0000000A
```

Indicates that an instruction completed execution and wrote a value to a register.

Memory Store

```
PC=FFFF0010 IR=34C70000 ST A=00000000 D=0000001E
```

ST indicates a store operation

A (Address) is the memory address written to

D (Data) is the value stored

Halt Condition

```
HALT
```

Indicates that the CPU has reached a halt instruction and execution has completed.

Final Register Dump

```
r00=00000000
r01=00000000
...
```

After a halt condition, the full architectural register state is emitted to allow verification of final program results.

Formatting Design

The output format was designed with the following considerations:

- **Readability:** Human-readable text allows immediate inspection using standard serial terminals
- **Consistency:** Fixed-width hexadecimal fields simplify parsing and comparison
- **Compactness:** Only relevant fields are emitted per event to reduce bandwidth usage
- **Determinism:** Each line corresponds to a discrete execution event, making traces easy to follow sequentially

How to Use

The debug output is transmitted over UART and can be viewed using a serial terminal such as PuTTY (preferred).

Setup

Open PuTTY

Under **Connection Type**, select **Serial**

Set the following parameters:

- **Serial line (COM port):** Select the appropriate port
 - Example: COM3 (can be identified using Device Manager)
- **Speed (baud rate):** 115200

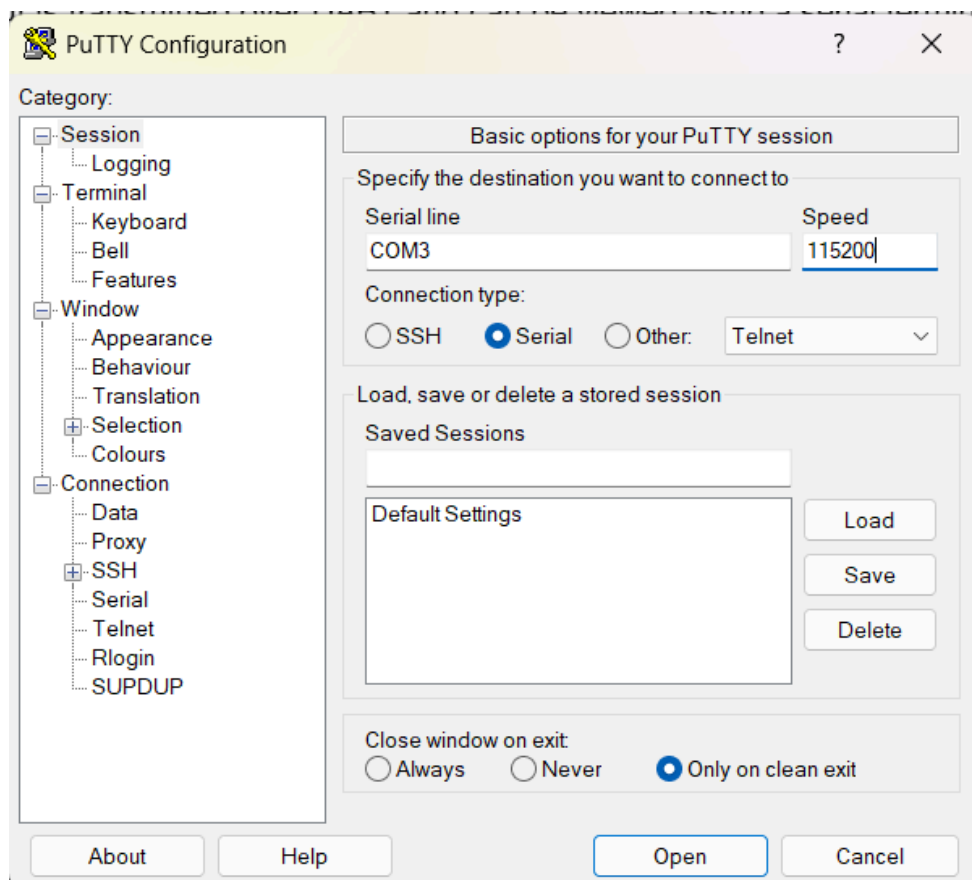


Figure 1: Session Configuration Settings

Serial Configuration

Ensure the following settings are configured (default 8N1):

- **Data bits:** 8
- **Stop bits:** 1
- **Parity:** None
- **Flow control:** None

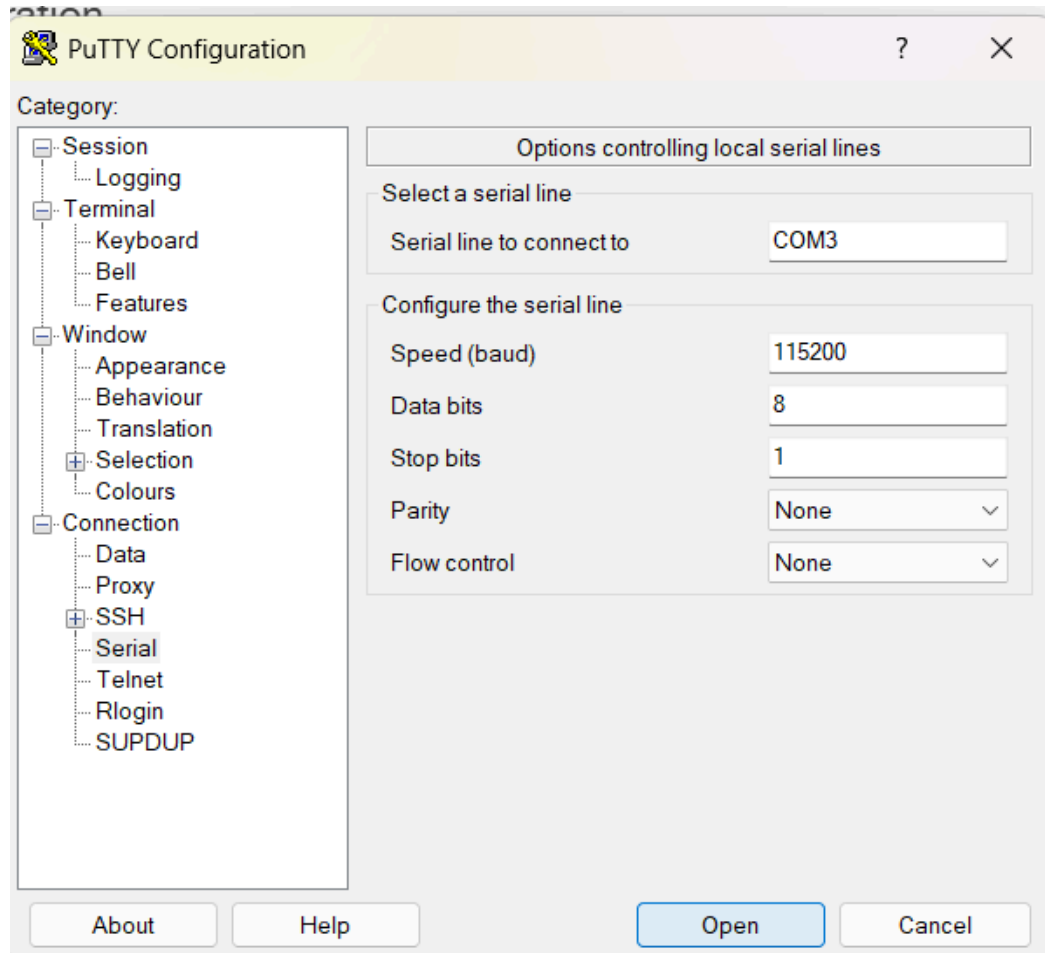


Figure 2: Serial Configuration Settings

Running the System

1. Program the FPGA with the CPU design
2. Open the configured PuTTY session
3. Reset or start the CPU
4. Observe the UART output in real time

The terminal will display execution trace lines corresponding to register writes, memory operations, and final CPU state upon halt.


```

COM3 - PuTTY
PC=FFFF0000 IR=0480000A RD=04 WB=0000000A
PC=FFFF0004 IR=04A00014 RD=05 WB=00000014
PC=FFFF0008 IR=00C42800 RD=06 WB=0000001E
PC=FFFF000C IR=04E00100 RD=07 WB=00000100
PC=FFFF0010 IR=34C70000 ST A=00000000 D=0000001E
PC=FFFF0014 IR=29070000 RD=08 WB=0000001E
HALT
r00=00000000
r01=00000000
r02=00000000
r03=00000000
r04=0000000A
r05=00000014
r06=0000001E
r07=00000100
r08=0000001E
r09=00000000
r10=00000000
r11=00000000
r12=00000000
r13=00000000
r14=00000000
r15=00000000
r16=00000000
r17=00000000
r18=00000000
r19=00000000
r20=00000000
r21=00000000
r22=00000000
r23=00000000
r24=00000000
r25=00000000
r26=00000000
r27=00000000
r28=00000000
r29=00000000
r30=00000000
r31=00000000

```

Figure 3: Debug Output from test_mem.hex

Notes

- If no output is observed, verify the correct COM port and baud rate
- Ensure the FPGA UART is properly connected to the host system
- Resetting the CPU will restart the trace output